A Deep Learning Approach for Fruit and Vegetable Image Classification

Zin You Tan
Faculty of Computing
University Teknology Malaysia(UTM)
Johor, Malaysia
tanzinyou@graduate.utm.my

This paper presents a deep learning approach for the automated classification of fruits and vegetables using a custom convolutional neural network (CNN). The proposed model was trained and evaluated on a dataset comprising 3,825 images categorized into 36 distinct classes. The network architecture, designed specifically for this multi-class image recognition task, effectively extracts and utilizes image features to achieve high classification accuracy. On the independent test set, the model attained a test accuracy of 96.94%, demonstrating strong generalization to unseen data. Detailed analysis of the classification report and confusion matrix highlights the model's strengths and areas for improvement. The results indicate the potential of CNN-based methods for practical applications in retail, food industry automation, and mobile applications for grocery identification.

Keywords—image classification, convolutional neural network, deep learning, fruits, vegetables, computer vision, multi-class classification, automated recognition

I. Introduction

Image recognition, especially in the domain of fruit and vegetable recognition, is extremely challenging due to the appearance variations caused by varying lighting, photo angle, scaling and occlusion. The classification of these items can be used to extract valuable information which results in many practical applications, especially in the food and retail domains. For example, imagine an application that lets people take a picture of their groceries and receive a result suggestion based on what the groceries are. Likewise, in smart shopping environments such as Amazon Go, the checkout process can be made more efficient through the use of image recognition systems to track and categorize the item as they are selected by customers.

To tackle this, a custom deep convolutional neural network (CNN) architecture "Figure 1" is considered for the task of image classification for fruits and vegetables. This architecture was developed to efficiently learn and recognize images. This allows for efficient computation and high accuracy.

This CNN model is intended to serve as a strong backbone for extracting low and high-level image features required for good performance in a multi-class classifier of fruits and vegetables. This project aims to develop a deep learning model to support different business applications such as improving user experience on mobile apps or increasing the accuracy of image recognition system for a smart store.

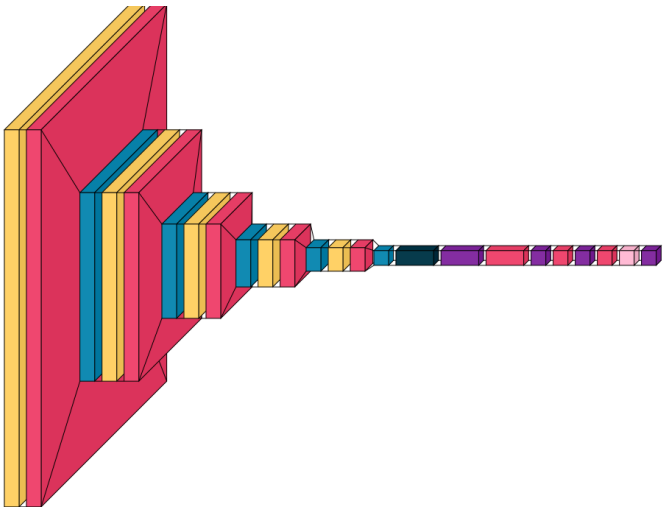

Fig. 1. Custom Convolutional Neural Network

II. Model Architecture

In this project, a custom CNN model was developed for multi-class image classification problem as shown in "Figure 1". The architecture consists of five sequential convolutional blocks and each of it is intended to learn the features hierarchy in the input images. Each block consists of a convolutional layer applying ReLU activation to ensure that non-linearity is captured, batch normalization for stabilization and speeding up training, and max pooling to reduce spatial dimensions and focus on the most important features.

The first and the second convolutional layers have 128 filters which improves the model's capability for detecting complex patterns and textures in images. In the next layer the number of filters is decreased to 64, which allows for a more narrow extraction of features, while controlling computational load. The fourth layer has 128 parameters, the fifth layer has 32 parameters for a good trade-off between feature extraction and efficiency. After the convolutional layers, the results are flattened and fed to three fully connected (dense) layers. The

first dense layer consists of 512 units and is succeeded by a batch normalization and a dropout to generalize and avoid overfitting. Following dense layer has 128 units, and the third one has 64 units, each followed by a batch normalization and dropout layer to further improve model stability. The final output layer is used with a "softmax" activation function to generate a probability distributions over the 36 target classes. Finally, the model was compiled with the Adam optimizer, "categorical_crossentropy" was applied for multi class classification task, and the model performance evaluation metric is choose by accuracy.

TABLE I. CUSTOM CNN MODEL ARCHITECRURE

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 128)	3,584
batch_normalization (BatchNormalization)	(None, 128, 128, 128)	512
max_pooling2d (MaxPooling2D)	(None, 64, 64, 128)	0
conv2d_1 (Conv2D)	(None, 64, 64, 128)	147,584
batch_normalization_1 (BatchNormalization)	(None, 64, 64, 128)	512
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 128)	0
conv2d_2 (Conv2D)	(None, 32, 32, 64)	73,792
batch_normalization_2 (BatchNormalization)	(None, 32, 32, 64)	256
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 64)	0
conv2d_3 (Conv2D)	(None, 16, 16, 128)	73,856
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 128)	512
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 128)	0
conv2d_4 (Conv2D)	(None, 8, 8, 32)	36,896
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 32)	128
max_pooling2d_4 (MaxPooling2D)	(None, 4, 4, 32)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 512)	262,656
batch_normalization_5 (BatchNormalization)	(None, 512)	2,048
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 128)	65,664
batch_normalization_6 (BatchNormalization)	(None, 128)	512
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
batch_normalization_7 (BatchNormalization)	(None, 64)	256
dropout_2 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 36)	2,340

To improve training efficiency and model performance, a few parameter tuning were included. The ReduceLROnPlateau parameter was used to reduce the learning rate by a factor of 0.3 if the validation loss did not decrease for 5 epochs in a row with a lower bound of 1e-6(0.000001). This strategy works well because the model does not get stuck in local minima. Furthermore, at the end of each epoch, it will stored the model weights that achieved the best validation accuracy during training using the ModelCheckpoint callback, so that the final model represent on the best possible performance to the test set data..

Data preprocessing and augmentation strategies were also designed carefully to enhance the model generalization. The image dimension has been resize to (128x128) for all the dataset. No data augmentation including rotation, flipping or zooming was performed in this project because the data is well balance. Images in training, validation and test set were rescaled and normalize the pixel value to [0, 1] by dividing a values of 255. The images are grouped into batches of size 16, and the labels are provided in categorical format which is suitable for multi class classification. This is useful for fair evaluation of the model in unseen examples and to avoid data leakage during evaluation.

III. MODEL IMPLEMENTATION

The dataset used contains 3825 images from 36 fruit and vegetables categories. Particularly, 3115 images were used as training set, 351 images used for validation set, and 359 images used for testing set. Each set of image data contains all 36 label classes to ensure a fair evaluation. The balance image distribution between classes allows for strong model training and evaluation of classification performance on the model.

The custom model was trained with a batch size of 16 using the training set generator. Validation of model performance was measured after each epoch with a separated validation data generator. Multiple parameters tuning and callbacks were used for enhancing training accuracy and avoiding the model overfitting. Some callbacks that used were learning rate reduction on plateau and model check point which is used to store the best performing model based on the metric "val_accuracy". All training and validation images were rescaled to the range [0, 1] before being fed into the model. The training was implemented with the Adam optimizer, and model accuracy was used as the primary evaluation metric.

CNN model was trained for 70 epochs and training accuracy, validation accuracy and loss were evaluated at every single epoch. The ReduceLROnPlateau callback was dynamically adjusting the learning rate when the validation loss was not improving anymore, and the learning rate at epoch 70 was dropped to 8.1000e-06(0.0000081). The ModelCheckpoint function is called to save the model weights associated with the highest validation accuracy obtained during training which is achieve at val_accuracy = 0.9601. The best model was then loaded on the test set. "Figure 2" and "Figure 3" shows the trend of training and validating accuracy and loss during the training epochs that both learning and convergence are effective.

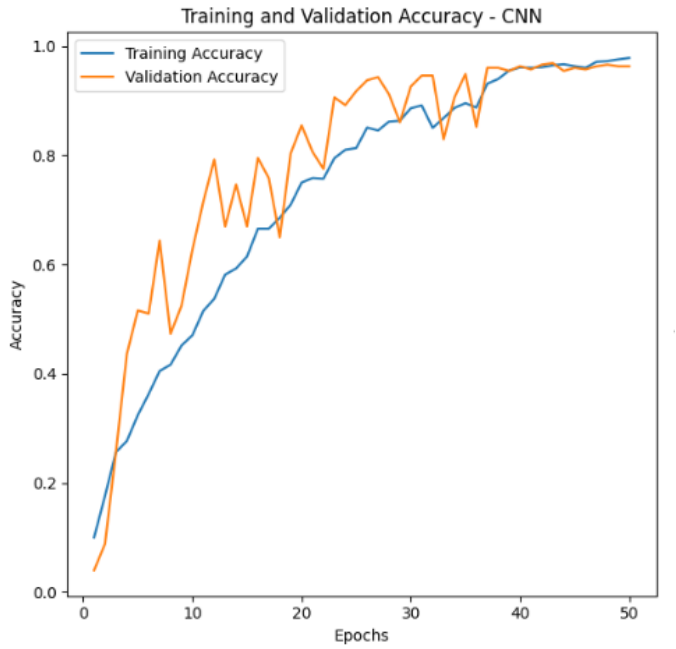

Fig. 2. Training and Validation Accuracy Over Epochs

Fig. 3. Training and Validation Loss Over Epochs

IV. Result and Discussion

The best performing model was chosen in the training phase based on validation accuracy and this model was used to test in the testing set. The model has an outperform result which the test accuracy was achieved 96.94% and a test loss of 0.158. This result showing a good generalization capability on unseen data.

In "Table II", precision, recall, F1-score for each class is provided in the classification report. Most of the classes have ideal or near-ideal scores. Classes such as beet root, cabbage, carrot, cauliflower, cucumber, eggplant, garlic, jalepeno, kiwi, lemon, lettuce, mango, paprika, pear, peas, pineapple, raddish, soy beans, spinach, tomato, turnip and water melon achieved F1-score of 1.00, which proving the model have a good classification performance. The majority of the classes performed well overall only a few classes had lower recall (<0.90), such as apple, banana, and sweetcorn. This indicating there was some misclassifications made by the model.

TABLE II. CLASSIFICATION REORT

	Classification Report:			
	precision	***recall***	***f1-score***	***support***
apple	1	0.7	0.82	10
banana	1	0.78	0.88	9
beetroot	1	1	1	10
bell pepper	0.82	0.9	0.86	10
cabbage	1	1	1	10
capsicum	0.91	1	0.95	10
carrot	1	1	1	10
cauliflower	1	1	1	10
chilli pepper	1	0.9	0.95	10
corn	0.83	1	0.91	10
cucumber	1	1	1	10
eggplant	1	1	1	10
garlic	1	1	1	10
ginger	0.83	1	0.91	10
grapes	0.91	1	0.95	10
jalepeno	1	1	1	10
kiwi	1	1	1	10
lemon	1	1	1	10
lettuce	1	1	1	10
mango	1	1	1	10
onion	0.91	1	0.95	10
orange	0.91	1	0.95	10
paprika	1	1	1	10
pear	1	1	1	10
peas	1	1	1	10
pineapple	1	1	1	10
pomegranate	0.91	1	0.95	10
potato	1	0.9	0.95	10
raddish	1	1	1	10
soy beans	1	1	1	10
spinach	1	1	1	10
sweetcorn	1	0.8	0.89	10

	Classification Report:			
	precision	***recall***	***f1-score***	***support***
sweetpotato	1	0.9	0.95	10
tomato	1	1	1	10
turnip	1	1	1	10
watermelon	1	1	1	10
accuracy			0.97	359
macro avg	0.97	0.97	0.97	359
weighted avg	0.97	0.97	0.97	359

To illustrate more on the model performance, the normalized confusion matrix is presented in "Figure 4". The majority of the predictions lie along the diagonal, demonstrating the high precision of the model across all classes. Only a few misclassifications are observed, and some confusion occurs near visually similar classes. The overall misclassification rate was 3.06% (11/359 test images misclassified). In general, the result shows that the model is very efficient at classifying 36 fruit and vegetable classes in the dataset.

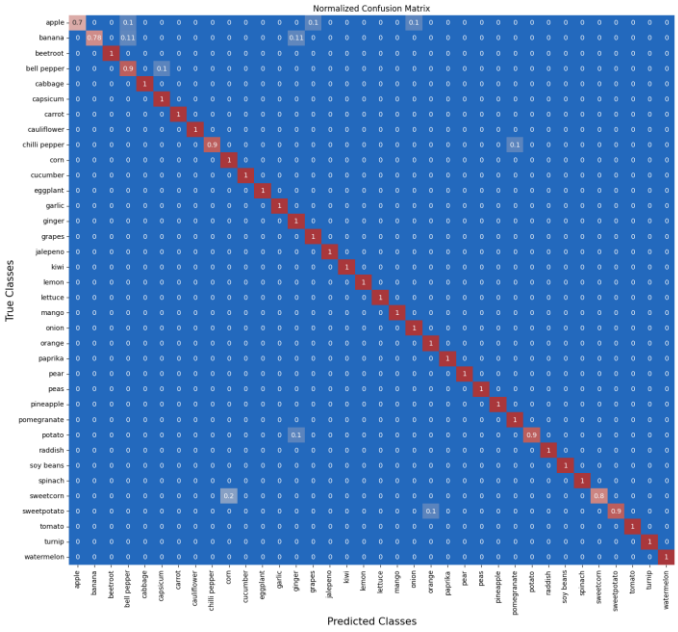

Fig. 4. Confusion Matrix of 36 Fruit and Vegetable Categories

Although the model has shown good performance on classification but there is some limitations that need to be aware of. The reported test result was achieved on a small held out set which only 359 images, and this relatively low number of samples may not comprehensively cover the diversity found in real world application. This study only does pixel value normalization as preprocessing, but no data augmentation such as rotation, flip and zoom, which may weaken the generalization of the model. According to the confusion matrix, the majority of the misclassification was between visually similar classes (e.g. both sweetcorn and banana are yellow and have long stick shape). Future work may include adding data augmentation methods and increasing the size of the dataset to enhance the model robustness and generalizability.

V. Conclusion

A custom Convolutional Neural Network (CNN) based approach is presented to classify 36 different types of fruits and vegetables in this project. The model achieved a test accuracy of 96.94% showing good results on test data. The classification report and confusion matrix show that the model is capable of well classifying most classes, and only a small number of misclassifications between some visually similar categories.

Despite these impressive results, there are still some weaknesses of this custom CNN model. For instance, there are sometimes confusions among appearance similarity of some classes. In this project, pixel value normalization as data preprocessing were used and did not employ any data augmentation. In the future, we may refine the performance through more complex data augmentation strategies like random rotations, flips, shifts or zooms and getting more diverse train data or explore more complex model structures.

In summary, the result of this project proves how CNN models are effective for the multi-class classification of images and becomes a good basis for future improvements in automatic fruit and vegetable recognition.